Co1AT1

1. Water jug problem

Represent each state as (amount in the 4-gallon jug, amount in the 3-gallon jug). Start at (0, 0) and use these actions:

1. Fill the 3-gallon jug: $(0, 0) \rightarrow (0, 3)$
2. Pour it into the 4-gallon jug: $(0, 3) \rightarrow (3, 0)$
3. Fill the 3-gallon jug again: $(3, 0) \rightarrow (3, 3)$
4. Pour from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full. One gallon remains in the 3-gallon jug: $(3, 3) \rightarrow (4, 2)$
5. Empty the 4-gallon jug: $(4, 2) \rightarrow (0, 2)$
6. Pour the remaining 2 gallons into the 4-gallon jug: $(0, 2) \rightarrow (2, 0)$

The 4-gallon jug now contains exactly 2 gallons.

2. Mars Rover as an intelligent agent

i. Percepts: Images and video; camera and instrument readings; position, orientation, and speed; terrain and obstacle information; and results from analysing soil or rock samples.

ii. Operating environment: Mars's terrain and atmosphere, changing light and weather, obstacles, communication delays with Earth, limited power, and the rover's finite onboard supplies. It is partially observable, dynamic, sequential, and resource-constrained.

iii. Actions: Move or turn; stop; aim or operate cameras; use an arm or drill to collect samples; analyse samples with onboard instruments; transmit data; and manage power.

iv. Performance measures: Safe operation, useful scientific discoveries, successful sample collection and analysis, progress toward mission goals, effective use of power and time, and reliable data transmission.

v. Suitable architecture: A model-based, goal-directed agent with utility-based planning is a good fit. The rover must track its internal state because it cannot observe everything directly, plan actions toward scientific goals, and weigh the value of an action against risks such as terrain hazards and limited power. It should also respond to urgent hazards without waiting for instructions from Earth.

3. 8-Queens problem

State: A partial or complete arrangement of queens on an 8×8 chessboard.

Initial state: An empty board.

Actions: Place a queen in an unoccupied square that is not attacked by any queen already placed.

Goal test: Eight queens have been placed, with no two in the same row, column, or diagonal.

Path cost: If the task is simply to find a solution, each placement can have cost 1; all solutions then have the same total cost.

A practical search strategy is *backtracking* : place one queen per column, try safe rows, and back up whenever no safe row remains. This avoids exploring placements that already conflict. The configuration shown in the assessment is not a solution because its first and last queens share a diagonal.

4. OLA cab booking agent

A goal-based problem-solving agent can help the customer choose a cab and route. The goal is to reach the destination, while considering cab type, estimated fare, travel time, availability, and the customer's preferences. A utility-based agent could rank valid choices when the customer wants to balance cost and comfort.

```
function ChooseCabAndPlan(origin, destination, preferences):
    options ← GetAvailableCabTypes(origin)

    bestPlan ← none
    bestScore ← negative_infinity

    for each cab in options:
        route ← FindRoute(origin, destination, cab)
        if route does not exist:
            continue

        fare ← EstimateFare(cab, route)
        time ← EstimateTravelTime(route)
        comfort ← ComfortScore(cab, preferences)
        availability ← CheckAvailability(cab)

        if availability is false:
            continue

        score ← Evaluate(fare, time, comfort, preferences)

        if score > bestScore:
            bestScore ← score
            bestPlan ← (cab, route, fare, time)

    if bestPlan is none:
        return "No suitable cab is available"

    return bestPlan
```

5. Uniform Cost Search

The PDF introduces a weighted graph for this question, but the graph itself—including its edges and travel costs—is not visible. UCS expands the unfinished route with the lowest accumulated cost first, so I need the graph to identify the actual least-cost path from S to G.